

Full-Stack Engineering Assessment

A real-world engineering task built on an existing system. Implement features across the full stack — progressively, stage by stage.

React 19 + TypeScript

React Query + Axios

.NET 8 + Dapper

PostgreSQL

4 stages · Stages are independent — complete them in any order. Stage 1 (Scraper) is optional and can be skipped if needed.

INTRODUCTION

Welcome to the UltraStore Assessment

You are joining the engineering team working on UltraStore — a product catalogue platform. The system exists and has been partially built. Your task is to implement the missing pieces, in the order they would naturally be built in production.

SYSTEM ARCHITECTURE



ALREADY IN PLACE

- ✓ Database schema (PostgreSQL)
- ✓ API controller signatures and routing
- ✓ Repository layer with UPSERT method
- ✓ TypeScript type definitions
- ✓ React Query + React Router configured
- ✓ Axios client + Vite proxy to `api.ultra.local`
- ✓ Component structure and styles
- ✓ 1 000 seed products in Store.API

YOUR TASK

- 1 Build the data scraper — collect all products and persist them to the local database
- 2 Implement backend filtering, pagination, and input validation
- 3 Build the frontend — product listing, filtering, and detail page
- 4 Performance improvements and scalability discussion

No required order — stages are independent and can be completed in any sequence. Stage 1 (Scraper) is optional and can be skipped if needed.

RUNNING SERVICES

SERVICE	URL	NOTES
Store.API	<code>http://store.ultra.local</code>	Source data — 1 000 seeded products. Read-only, do not modify.
UltraStore.API	<code>http://api.ultra.local</code>	The backend you will implement. Reads from <code>scraper.db</code> .
React Dev Server	<code>http://localhost:5173</code>	Frontend. Vite proxies <code>/api/*</code> to <code>api.ultra.local</code> automatically.

STAGE 1 OF 4

Scraper & Data Collection

Build the data pipeline: collect all products from Store.API and persist them to the local database.

01

Requirements

- 1 Implement `ScrapeAsync()` (C#) or `scrape()` (TypeScript). The method stub already exists — fill in the logic.
- 2 Extract all of the products from the store.
- 3 Send all of the collected products to the backend for validation and storage.

Note: Stage 1 is optional and can be skipped if needed.

STAGE 2 OF 4

Backend Implementation

Implement server-side filtering and pagination in the repository.

02

Part A — Repository: Filtering & Pagination

- 1 Filter products by search term (matches name and description), category, and price range.
 - 2 Return paginated results along with the total count of matching products.
- + **BONUS** Return `ProblemDetails` (RFC 7807) instead of plain strings for validation errors
 - + **BONUS** Suggest and implement at least one improvement to support scaling to millions of products

Frontend Implementation

Connect the React application to the API using React Query, implement filtering and search, then build a product detail page with client-side routing.

03

Part A — API Integration & Product Listing

- 1 Configure the Axios instance in `src/api/client.ts` with `baseUrl: '/api'`. Vite proxies `/api` to `api.ultra.local`.
- 2 Implement `useProducts(filters)` and `useCategories()` in `products.ts` using `useQuery`. The query function must call the Axios client and return the response data.
- 3 Render the product grid in `ProductsPage.tsx` using `useProducts()`. Show loading and error states. Populate the category dropdown from `useCategories()`.

Part B — Filtering & Search

- 4 Pass all active filter values as query parameters to `GET /api/products`.
- 5 Wire up the search input, category dropdown, and min/max price inputs. Reset the page number to 1 whenever any filter changes.

Part C — Product Detail Page

- 6 Implement `useProduct(id)` using `useQuery`.
- 7 Build the `ProductDetailPage`: extract the product ID from the URL with `useParams()`, fetch the product, render all its fields, and handle loading and error states.

- + **BONUS** Debounced search: fire the query only after 350 ms of inactivity to avoid a request on every keystroke
- + **BONUS** URL sync: persist filter state in the URL with `useSearchParams` so reloading or sharing a link restores the same filters
- + **BONUS** Context API: refactor filter state into `ProductContext` (scaffolded at `src/context/ProductContext.tsx`) to eliminate prop drilling

STAGE 4 OF 4

Improvements & Scalability

A small implementation task followed by an open architectural discussion.
Demonstrate that you can reason about performance and trade-offs beyond the

Reveal after Stage 3. May be explored as a discussion if implementation time is limited.

04

APPROACH

Think out loud. Trade-offs matter more than naming every technology.

Part B — Architecture Discussion

Imagine UltraStore needs to scale to 10 million products with 50 000 daily users. Walk through how you would evolve each area below.

Search	When would you replace SQL LIKE with a full-text search engine? What are the trade-offs of PostgreSQL FTS vs Elasticsearch?
Pagination	When does offset pagination break down at scale? How would you migrate to cursor-based pagination without breaking existing clients?
Caching	What would you cache and where — HTTP Cache-Control, Redis, or a CDN? Which endpoints benefit most?
Scraper at Scale	The scraper currently runs once. At scale it needs to run continuously. Scheduled job, event queue, or change-data-capture?
Horizontal Scaling	Multiple API instances behind a load balancer — what changes? What about running the scraper on multiple nodes simultaneously?
Database	Should you keep SQLite at this scale? When would you move to PostgreSQL, introduce read replicas, or consider sharding?

There are no right answers here. A candidate who proposes one approach and clearly explains its trade-offs is more valuable than one who lists every technology without conviction.